Document Number: P2588R0
Date: 2022-05-13
Reply to: Gonzalo Brito Gadeschi <gonzalob _at_ nvidia.com (http://nvidia.com)>
Authors: Gonzalo Brito Gadeschi, Eric Niebler, Anthony Williams, Thomas Rodgers
Audience: LEWG, Concurrency

`barrier` 's phase completion guarantees

Abstract

Unintended consequences of `std::barrier` 's specification constrain implementations to run the `CompletionFunction` on the last thread that arrives at the barrier during the phase. This prevents `std::barrier` from benefiting from hardware acceleration for thread synchronization. Removing these constraints is a *breaking change*. This paper aims to find a sweet spot for the barrier specification that delivers the functionality that applications need while allowing efficient implementations.

Introduction

The specification of `std::barrier<CompletionFunction>` requires the phase completion step to run when the expected count becomes zero on one of the threads that arrived at the barrier during the phase thread.barrier.class-1.2 (http://eel.is/c++draft/thread.barrier#class-1.2):

> When the expected count reaches zero, the phase completion step is run. For the specialization with the default value of the `CompletionFunction` template parameter, the completion step is run as part of the call to `arrive` or `arrive_and_drop` that caused the expected count to reach zero. For other specializations, the completion step is run on one of the threads that arrived at the barrier during the phase.

The specification of `std::barrier` does not require any thread that arrives at the barrier to call `std::barrier::wait`, but calling `barrier::wait` is necessary to observe phase completion thread.barrier.class-3.sentence-3 (http://eel.is/c++draft/thread.barrier#class-3.sentence-3):

> [...] the behavior is undefined if any of the barrier object's member functions other than `wait` are called while the completion step is in progress.

A thread that never calls `wait` can still arrive at the barrier again via synchronization through some other thread that does call `wait`. This is very useful in practice, as the following example shows:

Example 0: a producer / consumer pipeline (godbolt (https://godbolt.org/z/T8Y7494W5))

```
std::barrier<CF0> b0(2, cf0);
std::barrier<CF1> b1(2, cf1);

void thread_0() {
    while(true) {
        produce_data();        // A
        b0.arrive();           // B: signal data produced
        b1.arrive_and_wait();  // C: wait on data consumed
    }
}

void thread_1() {
    while(true) {
        b0.arrive_and_wait();  // D: wait on data produced
        consume_data();        // E
        b1.arrive_and_wait();  // F: signal data consumed
    }
}
```

In this example, `thread_0` at “A” produces some data, and then at “B” signals `thread_1` that the data is ready. `thread_0` will never wait on barrier `b0`. Then `thread_0` waits on `thread_1` consuming the data, and proceeds to generate new data.

The `[[nodiscard]]` attribute on `arrive` shows that `barrier` designers were not expecting threads to arrive at the barrier without calling wait.

The current standard wording in thread.barrier.class-1.2.sentence-3 (https://eel.is/c++draft/thread.barrier#class-1.2.sentence-3):

> For other specializations, the completion step is run on one of the threads that arrived at the barrier during the phase.

aims to provide implementations with enough freedom to run the `CompletionFunction` on any thread that participates in the barrier during the phase.

Unfortunately, the current wording requires all implementations to run the `CompletionFunction` as part of the call to `arrive` performed by the last thread that arrives at the barrier during the phase, as the following example shows:

Example 1: Guarantee that `CompletionFunction` runs if no thread waits (godbolt (https://godbolt.org/z/41vqEh58e))

```
std::barrier<CF> b{2, cf};
using tok_t = decltype(b.arrive());

void thread() {
    new tok_t(b.arrive());       // A: arrive and leak token
}                                // B: thread exit

auto t0 = std::thread(thread);  // C: Spawn two threads
auto t1 = std::thread(thread);

t0.join();                       // D: Join them
t1.join();

// E: Standard guarantees that CompletionFunction did run
```

This example spawns two threads at “C”, both of which execute the same sequence of operations:

- “A” arrive at the barrier and leak the token, and
- “B” exit.

After joining both threads at “D”, they no longer exist.

The standard guarantees in thread.barrier.class-1.2.sentence-1 (https://eel.is/c++draft/thread.barrier#class-1.2.sentence-1) that the completion function runs:

> When the expected count reaches zero, the phase completion step is run.

in one of the threads that arrived at the barrier during the phase (thread.barrier.class-1.2.sentence-3 (https://eel.is/c++draft/thread.barrier#class-1.2.sentence-3)). At “E”, these threads do not exist anymore. Therefore, the `CompletionFunction` must have run before “E”, and more precisely, it must have run before the last thread that arrived at the barrier during the phase exits.

That is: there is only one place in which it makes sense for conforming standard library implementations to run the phase completion step: as part of the `arrive` performed by the last thread that arrives at the barrier during the phase.

This is an unintended consequence of the interaction between:

- the freedom for threads to never call wait, and
- the guarantees about when and where the phase completion step runs.

These consequences are problematic in practice, where Amhdahl’s law (https://en.wikipedia.org/wiki/Amdahl%27s_law) limits the scalability of massively parallel applications with small “serial” overheads on modern NUMA (https://en.wikipedia.org/wiki/Non-uniform_memory_access) architectures with millions of hardware threads. While `std::barrier` ’s split `arrive` / `wait` APIs enable threads to hide the cost of synchronization behind independent work, the amount of independent computation available in real applications is limited. Clauses like thread.barrier.class-1.2.sentence-3 (https://eel.is/c++draft/thread.barrier#class-1.2.sentence-3) aim to enable `std::barrier` to use hardware accelerators for synchronization, such as those available in NVIDIA GPUs, to allow applications to hide the cost of synchronization behind small amounts of independent work, but due to the unintended consequences explained above, implementations currently cannot do so.

Fixing this requires a breaking change. There are clear engineering trade-offs between varying degrees of guarantees which determine the set of well-formed programs with different degrees of implementation flexibility which determine performance.

This paper aims to to help the reader answer the question: What is the sweet spot for `std::barrier` semantics, and what are the consequences of the breaking changes required to get there?

The following section analyzes the semantics of the current barrier specification and the different changes that we could make to balance functionality and performance. Then, we evaluate changes according to the functionality provided, their ability to leverage hardware

acceleration, and the impact of a change on both existing and potential standard library implementations and end-user applications.

Finally, the authors suggest changes that balance functionality and performance to deliver the functionality that applications need in practice while allowing efficient implementations and propose wording for these changes.

Analysis of barrier semantics

The “core” semantics of the `std::barrier` phase completion step are:

1. The last arrival *happens-before* phase completion, which *happens-before* any thread unblocks from `wait`.
2. Establish *cumulativity* from all threads that arrive at the barrier during the phase to all threads unblocked from `wait` through the thread that runs the `CompletionFunction`.

These “core” semantics enable applications to, e.g., perform a reduction in a critical-section in-between arriving and waiting:

Example 2: reduction in critical section (godbolt (https://godbolt.org/z/qcTT15bEa))

```
std::vector<int> data(nthreads);
int reduction;

auto reduce = [&] {
  reduction = std::accumulate(data.begin(), data.end(), 0);
};
std::barrier<decltype(reduce)> b(nthreads, reduce);


void thread(size_t i) {
    data[i] = produce_data();  // A
    b.arrive_and_wait();       // B
    consume(reduction);        // C
}
```

Here, threads produce some data at “A” and arrive and wait at the barrier at “B”. `reduce` is then called inside a critical section, after the last thread arrives, and before any thread is unblocked from the wait. Since all threads participating in the barrier are “stalled” at the `wait`, `reduce` can access `data` safely without data-races. Finally, `reduce` happens-before any thread is unblocked from the `wait`. That is, all uses of `reduction` at `C` observe the value that `reduce` initialized it with safely and without data-races.

The main design choices that this paper concerns itself with are “when” and “where” should the C++ standard guarantee that the `CompletionFunction` runs. These two properties, “where” and “when”, are intertwined. The following Table 1 explores some of the main options, the choices they enable, and their impact on hardware acceleration, existing implementations, and user applications.

Table 1: Design tradeoffs: “When does the `CompletionFunction` run?”, “Where is it allowed to run (on which threads)?”, “Does the `CompletionFunction` run if no thread calls `wait` ?”, “What are the hardware acceleration opportunities of these constraints?”, “What’s the impact on implementations and users?”

When	Where	Runs if no thread waits?	Hardware acceleration opportunities	Implementation impact	User impac
Last `arrive`	Last thread to arrive	Yes	Very low	None	None
Any `arrive`	Any thread that arrives	Yes	Very low	None	None
Any `wait`	Any thread that calls `wait`	No	Low	All	None f any applica survey minima otherw
Any `arrive` or `wait`	A thread that arrives or waits	**Options**: “Yes” “No” “Unspecified” “Implementation defined”	Medium	If answer to “Runs if no thread waits?” is No, None. Otherwise, All.	None f any applica survey minima otherw
After last arrive before any thread unblocks from wait	**Options**: “Unspecified” “A thread that arrives or waits” “A new thread” “A thread that arrives or waits or a new thread”	**Options**: “Yes” “No” “Unspecified” “Implementation defined”	High	If answer to “Runs if no thread waits?” is No, None. Otherwise, All.	None f any applica survey minima otherw

Restricting the choice of “when” significantly constrains the threads in which it makes sense for implementations to run the `CompletionFunction`. For example, restricting “when” to particular API calls, including the broad “Any `arrive` or `wait`”, restricts implementations to only run the phase completion step within those API calls (on threads making these API calls).

An important question impacting Example 1 is whether the `CompletionFunction` runs if no thread waits. The third column in Table 1 answers this question for the options considered. For the last two rows in the table, the standard is free to define these semantics as it wishes. Some options are provided in the table inline, and vary from well-defined semantics (“Yes” or “No”), to “implementation-defined” or “unspecified”.

On hardware acceleration

The column on “Hardware acceleration opportunities” describes how effective “hardware barrier synchronization accelerations” can be at accelerating synchronization via `std::barrier`. Hardware accelerators that have more freedom to pick which thread completes the barrier phase (up to the freedom of creating a new thread to do so) can be significantly more effective than those that cannot make this choice. As the number of hardware threads available on computer hardware increases, the benefits of these accelerators outweighs their costs. NVIDIA GPUs have been shipping barrier accelerators for many years. Being descriptive and not prescriptive enables hardware vendors to innovate.

Impact on existing implementations

If the standard *allows* the `CompletionFunction` to run even if no thread ever waits, no mainstream standard library implementation (`libc++`, `libstdc++`, and MSVC STL) needs to change. Otherwise, these standard library implementations would need to change.

For the last two rows, selecting “implementation-defined” behavior would require the existing implementations to document their behavior.

Largest possible semantic change

To evaluate the impact of breaking changes, this section defines the “largest” possible semantic change we could make. It allows `CompletionFunction` to run:

- **When**: Runs after last arrive before any thread unblocks from wait (no other requirements about “when”)
- **Where**: Unspecified.
- **Runs if no thread waits?** Unspecified.

This change **breaks** three `std::barrier` guarantees:

- That the phase completion step is always run when the expected count reaches 0.
  - With this change, it is only guaranteed to run if at least one thread calls `wait`, but this change does not guarantee that it does not run if no thread calls `wait`. That is, even if

no thread calls `wait`, these change allows the implementation to run the `CompletionFunction`, but it does not require the implementation to do so.

- That the phase completion step runs on the last thread that arrived at the barrier during the phase.
  - With this change, the phase completion step could run on any thread (it's unspecified where it runs). It can run on the last thread that arrived, but it can also run on a thread that waits, or on a new thread.
- That a barrier with a default completion function can be arrived on indefinitely without calling wait.
  - With this change, this becomes undefined behavior.

Potential impact on implementations

Thomas Rodgers mentioned that libdispatch (https://developer.apple.com/documentation/dispatch) barriers potentially complete on a different thread in the thread pool: all the work in the queue before the enqueuing of the barrier work item completes before the barrier work item, whose completion may happen on some other thread. The largest change would enable such an implementation.

NVIDIA's barrier accelerators can run the phase completion step very close to the memory where the barrier resides. The largest change enables such an implementation.

Impact on existing applications

We surveyed all public uses of `std::barrier` on GitHub as well as within some NVIDIA's code bases that use them heavily.

The impact of all changes considered - including the largest change - on all existing applications we surveyed was **non-existent**. The feature is a new C++20 feature that's been only available since:

- GNU libstdc++: version 11.1, released 27 April 2021 (https://gcc.gnu.org/onlinedocs/libstdc++/manual/status.html#status.iso.2017)
- LLVM libc++: version 11, released 26 August 2020 (https://libcxx.llvm.org/Status/Cxx20.html)
- MSVC: version 2019 16.9, released 2nd March 2021 (https://docs.microsoft.com/en-us/cpp/overview/visual-cpp-language-conformance?view=msvc-170#c-standard-library-features)

and it is a relatively niche and sharp tool.

All applications we discovered were either educational or small toy programs for learning purposes. They all had one thread that both arrived and waited at the barrier during all barrier phases. None of them relied on a particular thread running the `CompletionFunction` (we explore the uses we can imagine in the next section).

Potential impact on applications

While the authors could not find any application in the wild that would break due to the largest change, together with the experts we polled, we can imagine some.

Portability concerns

Anthony Williams raised the following concern: if most standard library implementations run the phase completion step as part of the last thread that arrives at the barrier during the phase, applications might end up silently and accidentally relying on this guarantee. Therefore, applications will not be portable to implementations that do something else.

Count-down with effect

P2300 (https://wg21.link/P2300) `when_all` algorithm accepts a variable number of asynchronous tasks and executes a continuation when they all complete. Given `std::barrier` as specified today, one valid implementation of `when_all` simply arrives at the barrier from all asynchronous tasks. After the last task arrives, the continuation runs. No thread calls `wait`, and the barrier is never used again.

The largest change would silently break the semantics of such a `when_all` implementation, causing the application to hang on implementations that do not run the phase completion step as part of the last thread that arrives.

This implementation of `when_all` would be using `std::barrier` as a “single-use count-down with effect”: a counter initialized to the number of asynchronous tasks that each task decrements by `1`. When the counter reaches 0, it runs some function for its effect.

Since the counter is “single-use”, it would make more sense for `when_all` to use `std::latch` here. However, `when_all` cannot do so because `std::latch` does not support `CompletionFunctions`. A way to enable this use case would be to add support for `CompletionFunctions` to `std::latch`. Furthermore, since `std::latch` is single-use, the semantics that make sense for `std::barrier` do not necessarily make sense for `std::latch`. This is worth exploring.

Barriers and latches are synchronization primitives intended to synchronize groups of threads with each other. However, this is not what the `when_all` example needs. The `when_all` example synchronizes a set of threads with the thread that runs some effect; it does not synchronize the threads themselves with each other. So another alternative that might be worth exploring is to provide such synchronization primitives in the standard library.

Thread id

Anthony Williams mentioned that they commonly see applications in which a handle, like a database handle, is stored in `thread_local` storage (TLS). Such an application could use the knowledge that the `CompletionFunction` is only executed by a thread that arrived at the barrier during the phase to just access this handle from the `CompletionFunction`.

Thomas Rodgers recognized that such an application could, in some cases, store a `shared_ptr<Handle*>` in the `CompletionFunction` itself and access the database handle through it. However, if the database handle is tied to the thread that created it, this approach would not work. He also recognized that on the libdispatch (https://developer.apple.com/documentation/dispatch) barrier model, such an application could not depend on TLS either.

This is an example of a larger class of applications that rely on the guarantee that one of the threads that arrived at the barrier during the phase runs the `CompletionFunction`. Thread locals are one example; inspecting the thread id and doing something with it would be another.

Any change that relaxes the guarantees about “where” (on which thread) the `CompletionFunction` may run will be turning some of these use cases from “well-formed” programs into non-portable or illegal programs. We can imagine these applications but have not found them in the wild.

Suggestion

Our suggestion is to pursue some flavor of the largest semantic change, restricting ourselves to the options that do not impact any existing standard library implementation or application we surveyed in the wild. While we can imagine theoretical applications that would be impacted by any option we take, it is very hard to write applications that do so correctly. Given the novelty and niche of this feature, we expect the impact on such applications to be minimal to non-existent. All experts we polled from the domains of applications using `std::barrier`, standard library implementors, and designers of `std::barrier` were surprised by the current semantics. These semantics were not intended. The intention was to enable `std::barrier` to leverage hardware acceleration. If feasible, our recommendation is to backport this change to C++20.

That is, our suggestion is to guarantee that:

- The last thread arriving at a barrier phase happens-before the `CompletionFunction` runs which happens-before any thread is unblocked from that phase.
- Cumulativity is established between all threads arriving at the barrier during the phase and the thread running the phase completion step, and between that thread and any thread that observes phase completion via a call to `wait`.
- If no thread observes phase completion, whether phase completion runs is *unspecified*.

- The phase completion step runs on one of the threads that arrived or waited at the barrier during the phase *or* a new thread.

Running the phase completion step on an “unspecified” thread *includes* existing unrelated threads, with code injected at any point.

Suggested wording

The proposed change modifies thread.barrier.class-1 (http://eel.is/c++draft/thread.barrier#class-1) as follows:

> Each barrier phase consists of the following steps:
>
> - (1.1) The expected count is decremented by each call to arrive or `arrive_and_drop`.
> - (1.2) When the expected count reaches zero, ~~the phase completion step is run. For the specialization with the default value of the CompletionFunction template parameter, the completion step is run as part of the call to `arrive` or `arrive_and_drop` that caused the expected count to reach zero. For other specializations, the completion step is run on one of the threads that arrived at the barrier during the phase.~~the phase completion step may run. The phase completion step shall run if at least one thread observes phase completion by waiting at the phase synchronization point. The thread on which phase completion runs is one of the threads that arrived or waited at the barrier during the phase or it is a new thread.
> - (1.3) When the completion step finishes, the expected count is reset to what was specified by the expected argument to the constructor, possibly adjusted by calls to `arrive_and_drop`, and the next phase starts.

Optional changes

Remove `[[nodiscard]]` attribute from `barrier::arrive`

As Example 0 () shows, discarding the return value of `barrier::arrive` is fine and useful. We should consider removing `[[nodiscard]]`.

Deprecate the `const` overload of wait and add non-`const` overload

Since wait can complete the barrier phase, we might want to consider deprecating the `const`-overload and adding a non-`const` overload.

This would require changing all implementations. To support the deprecated `const` overload, any implementation that wants to modify the barrier internal state during `wait` needs to use `mutable` anyways.

Acknowledgements

Everyone that helped, in particular, Olivier Giroux, Eric Niebler, David Olsen, Anthony Williams, and Thomas Rodgers.